FIAP ON

TRATORES DIGITAIS - HARDWARES PARA IA

EXPLORANDO A INTELIGÊNCIA ARTIFICIAL: O PAPEL DOS AGENTES INTELIGENTES

2

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Market share de IA no mundo. Note que Machine Learning ainda representa a maior fatia 7
Figura 2 - Chatbot simples no Colab 11
Figura 3 - Robô aspirador automático como exemplo de agente 15
Figura 4 - Drones possuem modelos de voos e estabilidade 26
Figura 5 - Carros autônomos da Tesla possuem modelos com regras de trânsito 26
Figura 6 - Toad, da saga Super Mario Bros 26
Figura 7 - Digit, o robô humanoide da Agility Robotics com recursos de mobilidade e manipulação 28
Figura 8 - Tela do ChatGPT e os joinhas como feedback para o aprendizado por reforço 32
Figura 9 - Estrutura de funcionamento de um agente de IA 40

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 - Comparativo entre os tipos de agentes de IA 37

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo de chatbot utilizando Python 13
Código-fonte 2 - Exemplo de agente 17
Código-fonte 3 - Parte do código do agente 18
Código-fonte 4 - Condição inicial dos ambientes no código 19
Código-fonte 5 - Condição após a primeira iteração 19
Código-fonte 6 - Condição após a terceira iteração 19
Código-fonte 7 - Condição após a quarta iteração 20

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 EXPLORANDO A INTELIGÊNCIA ARTIFICIAL: O PAPEL DOS AGENTES INTELIGENTES 6
1.1 Introdução 6
1.2 Diferença entre Agente, Chatbot e Automação 8
1.3 Exemplo de Chatbot 10
1.4 Exemplo de Agente Simples 15
1.5 Aplicações dos Agentes 20
2 CLASSIFICAÇÃO DE AGENTES 23
2.1 Agentes Reflexivos Simples 23
2.2 Agentes Baseados em Modelo 23
2.3 Agentes Orientados a Metas 27
2.4 Agentes Baseados em Utilidade 29
2.5 Agentes de Aprendizado 30
2.6 Multiagentes 33
3 COMPARATIVO 37
3.1 Resumo comparativo 37
4 O CHAT GPT POSSUI QUAIS TIPOS DE AGENTES? 38
5 ESTRUTURA DE FUNCIONAMENTO DE UM AGENTE DE IA 40
6 FERRAMENTAS 42
6.1 Llama 42
6.2 TensorFlow 42
6.3 PyTorch 43
6.4 OpenAI Gym 43
6.5 spaCy 43
6.6 NLTK (Natural Language Toolkit) 43
6.7 LangChain 44
6.8 Hugging Face Transformers 44
6.9 OpenCV 45
6.10 YOLO (You Only Look Once) 45
6.11 Resumo comparativo 45
CONCLUSÃO 46
REFERÊNCIAS 48

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 EXPLORANDO A INTELIGÊNCIA ARTIFICIAL: O PAPEL DOS AGENTES INTELIGENTES

1.1 Introdução

A discussão em torno da IA evoluiu dos chatbots, que são interfaces simples desenvolvidas para responder perguntas dos usuários, para agentes de IA mais avançados. Esses agentes, na verdade, são programas autônomos capazes de observar o “ambiente”, tomar decisões e realizar ações para atingir metas específicas.

Eles podem monitorar fluxos de dados, automatizar processos complexos e realizar tarefas sem necessidade de supervisão constante.

E o que seriam esses “ambientes”? Referem-se ao contexto ou sistema no qual o agente de IA (que é um programa de automação mais inteligente que um chatbot) está inserido e com o qual ele interage. Esse ambiente pode variar amplamente dependendo da aplicação que o desenvolvedor implementou e pode incluir:

a) Ambiente físico, onde sensores captam informações do mundo real, como temperatura, movimento, localização, e os agentes são utilizados em dispositivos IoT ou sistemas robóticos.

b) Ambiente digital, onde dados e informações provenientes de sistemas online, como mensagens de texto, serviços web, registros em bancos de dados, fluxos de dados em tempo real ou APIs, são coletados 24 horas por dia pelos agentes digitais. Aliás, qualquer agente é digital, porém, para o **ambiente físico**, definimos que o agente **também é físico**.

c) Ambiente simulado, que são cenários modelados em ambientes virtuais ou simulados, como jogos de console e computadores ou sistemas de testes controlados, como aviação, exploração de petróleo e medicina, temos os agentes que experimentam e aprendem em um ambiente simulado, portanto, agente simulado.

d) Ambiente híbrido, que corresponde à combinação de elementos físicos e digitais. Por exemplo, um agente que monitora sensores em uma fazenda e

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

analisa previsões meteorológicas vindas da Internet. Esse agente também é chamado de híbrido.

Portanto, o “ambiente” representa o conjunto de informações e estímulos que o agente observa para interpretar situações e tomar decisões alinhadas aos seus objetivos.

Com as empresas do mundo todo buscando soluções mais avançadas de automação, a popularidade desses agentes está em ascensão: o mercado de agentes de IA de 2024 foi estimado em US$ 184 bilhões, e, de acordo com o site Statista (<https://www.statista.com/outlook/tmo/artificial-intelligence/worldwide>), espera-se um crescimento anual de 28,46% de 2024 até 2030. O gráfico a seguir demonstra a projeção desse crescimento.

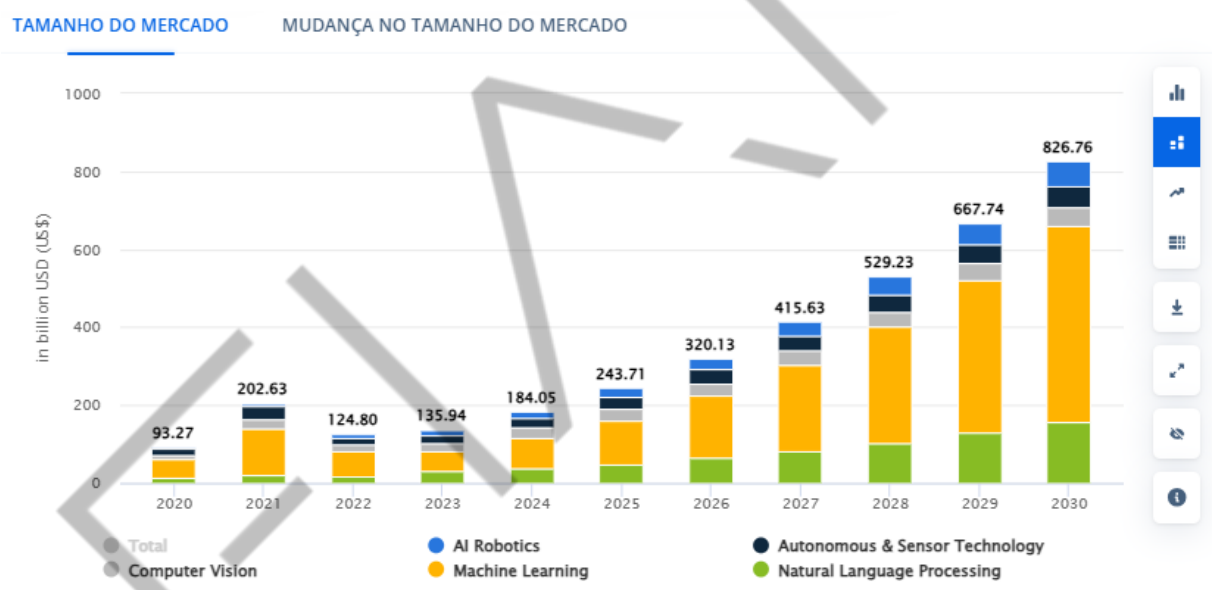

Figura 1 - Market share de IA no mundo. Note que Machine Learning ainda representa a maior fatia
Fonte: Statista (2024)

Esse crescimento é impulsionado pela crescente demanda por automação e pela busca por experiências mais personalizadas para os clientes. Por exemplo, o Sistema Único de Saúde (SUS) utiliza agentes de IA para automatizar tarefas do ciclo de receita, como verificação de elegibilidade para procedimentos médicos e gestão de reivindicações dos não atendidos, enquanto as equipes de desenvolvimento de software implementam agentes para detectar e diagnosticar automaticamente problemas de desempenho em suas aplicações.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

A Receita Federal do Brasil tem implementado diversas iniciativas baseadas em IA para aprimorar a detecção de fraudes nas declarações anuais do Imposto de Renda. Essas medidas são baseadas em agentes que visam identificar inconsistências e irregularidades de forma mais eficiente, garantindo maior conformidade tributária.

1.2 Diferença entre Agente, Chatbot e Automação

Como já vimos, a evolução da tecnologia tem ampliado o uso de sistemas baseados em IA e automação. No entanto, para compreender as diferenças entre chatbots, sistemas de automação e agentes inteligentes, começamos analisando suas características, funcionalidades e aplicações. Embora esses termos sejam frequentemente usados de forma intercambiável, cada um representa um nível distinto de sofisticação tecnológica e autonomia.

Os **chatbots** são interfaces desenvolvidas para interagir com os usuários por meio de texto (WhatsApp, Telegram, Facebook Messenger etc.) ou voz (algumas skills da Alexa e do Google Assistant, além de sistemas de atendimento telefônico), geralmente programadas para responder a perguntas específicas. Eles são amplamente utilizados em sistemas de atendimento ao cliente, como em e-commerces e redes sociais, onde respondem perguntas frequentes ou fornecem informações básicas.

O funcionamento de um chatbot tradicional é baseado em scripts ou árvores de decisão, isso significa que ele segue fluxos predefinidos de interação. Apesar de alguns chatbots modernos utilizarem técnicas de PLN (Processamento de Linguagem Natural) para entender melhor as mensagens, eles ainda são limitados a comandos predefinidos e não conseguem tomar decisões além do escopo de sua programação.

A **automação**, por outro lado, abrange sistemas que realizam tarefas repetitivas ou complexas sem a necessidade de intervenção humana constante. Um exemplo claro disso é a automação industrial, onde máquinas controlam processos de fabricação ou linhas de montagem. Outro exemplo é a automação de preenchimento de formulários em serviços Web. Essa prática nem sempre é bem vista, porque também são usados em automações maliciosas que sobrecarregam tais

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

serviços. Por esse motivo existe o CAPTCHA, que significa Completely Automated Public Turing test to tell Computers and Humans Apart (Teste de Turing Público e Automatizado para Diferenciar Computadores e Humanos).

Sistemas de automação são rigidamente programados para seguir fluxos predefinidos e não possuem a capacidade de aprender ou se adaptar a novas condições. Eles são ideais para tarefas previsíveis e de alta repetibilidade, como a geração de relatórios financeiros ou o acionamento de dispositivos em horários específicos. Enquanto os chatbots buscam interagir com usuários, os sistemas de automação têm foco na execução eficiente de processos internos.

Os **agentes inteligentes** representam um nível superior de sofisticação tecnológica. Diferentemente dos chatbots e dos sistemas de automação, os agentes inteligentes são autônomos e possuem a capacidade de observar o ambiente, tomar decisões e agir para atingir objetivos específicos.

Um agente inteligente pode combinar várias tecnologias, como Processamento de Linguagem Natural, Aprendizado de Máquina e sensores, para processar informações complexas, planejar ações e executar tarefas de maneira adaptativa.

Por exemplo, assistentes virtuais, como Alexa e Siri, não apenas respondem a perguntas, mas também aprendem com o comportamento do usuário dependendo da skill e do serviço adotados, adaptando-se às suas preferências ao longo do tempo. Eles também podem executar tarefas contextuais, como ajustar a temperatura de um termostato com base nas condições climáticas ou recomendar produtos com base no histórico de compras.

Uma diferença central entre essas três tecnologias é o nível de autonomia e aprendizado. Enquanto os chatbots e os sistemas de automação seguem regras rígidas, os agentes inteligentes utilizam o aprendizado de máquina para melhorar suas decisões com base em experiências anteriores.

Além disso, enquanto os chatbots interagem diretamente com os usuários e os sistemas de automação se concentram na execução de fluxos internos, os agentes inteligentes combinam as duas funções, interagindo com usuários e ambientes externos para tomar decisões mais complexas.

A escolha entre chatbots, automação e agentes inteligentes depende do objetivo e do contexto. Por exemplo, um e-commerce pode optar por um chatbot para

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

responder perguntas frequentes dos clientes, quais os contratos a serem vencidos, enquanto uma fábrica pode implementar um sistema de automação para aumentar a eficiência operacional. Em relação a um agente inteligente, ele seria mais indicado em situações que exigem tomada de decisão em tempo real, como monitoramento de segurança em ambientes dinâmicos.

Em resumo, a principal diferença está no grau de complexidade e da capacidade adaptativa. Os chatbots oferecem interações básicas, os sistemas de automação garantem eficiência em processos repetitivos, e os agentes inteligentes combinam aprendizado, autonomia e adaptabilidade para solucionar problemas complexos. E por isso que, essa compreensão, é fundamental para determinar qual tecnologia melhor atende às demandas de cada aplicação.

1.3 Exemplo de Chatbot

O chatbot no código Python a seguir é um dos mais simples, onde as perguntas e as respostas são fixas. Em uma aplicação real, ele teria centenas ou milhares de perguntas semelhantes com as mesmas respostas, para que seja possível atender as diversas formas do usuário solicitar uma mesma informação. Por exemplo: “qual o prazo de entrega” ou “quando o meu pedido chegará” possuem a mesma resposta. Você poderia fazer essa sequência de perguntas e respostas em um arquivo csv a parte para não poluir tanto o arquivo Python, montando uma espécie de banco de dados rústico local.

Era assim que os programadores faziam na popularização dos chatbots no WhatsApp em 2021. Passavam o dia atualizando a lista de perguntas à medida que o usuário não tinha sucesso com as respostas. O Twilio era um dos mais famosos serviços Web responsável em manter os bots de pé no WhatsApp.

Caso queira, pode copiar esse código e colar em um Colab vazio e, em seguida, executar a célula. Você também pode adicionar novas iterações seguindo o padrão de perguntas dado pelo código. Quando rodar a célula no Colab, terá um campo vazio abaixo do código (indicado pela seta vermelha na figura a seguir) para você digitar sua pergunta.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

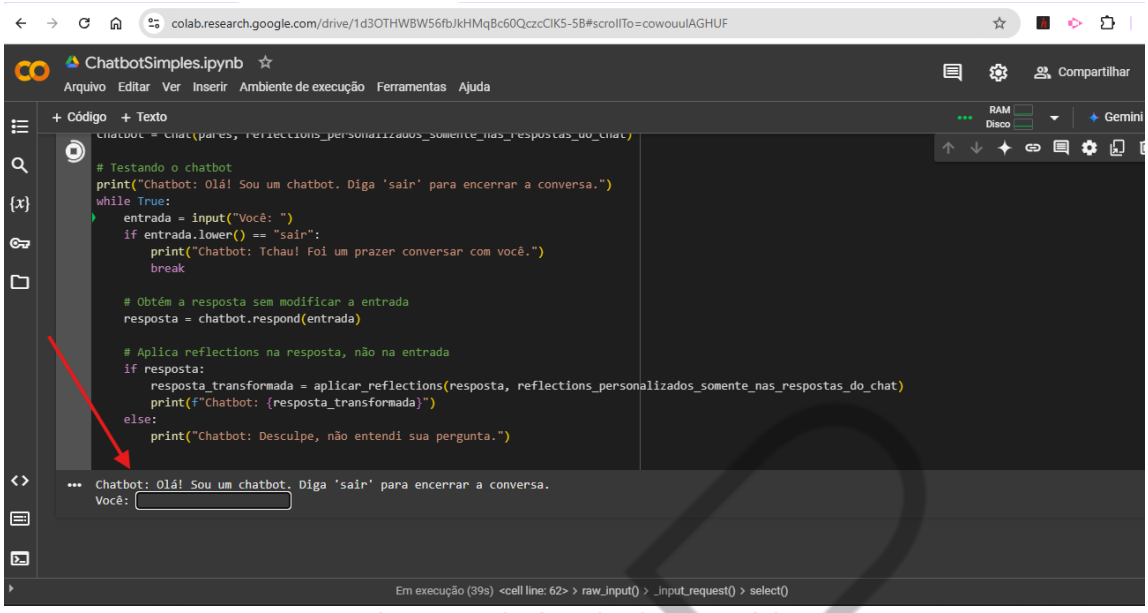

Figura 2 - Chatbot simples no Colab
Fonte: Elaborado pelo autor (2024)

```
import nltk
from nltk.chat.util import Chat

# Dicionário personalizado de reflections
reflections_personalizados_somente_nas_respostas_do_chat = {
    "eu": "você",
    "meu": "seu",
    "minha": "sua",
    "meus": "seus",
    "minhas": "suas",
    "seu": "meu",
    "sua": "minha",
    "seus": "meus",
    "suas": "minhas",
    "vou": "vai",
    "era": "fui"
}

# Pares de padrões e respostas
pares = [
    [
        r"meu nome é (.*)",  # Captura o nome do usuário
        ["Olá, %1! Como posso ajudar hoje ?"]
    ],
    [
        r"eu sou (.*)",   # Captura informações sobre quem o
usuário é
        ["Por que você é %1? Me conte mais."]
    ],
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    [
        r"meu (.*) é (.*)", # Captura algo que o usuário possui
        ["Por que meu %1 é %2? Isso parece interessante."]
    ],
    [
        r"como você está?", # Pergunta sobre o estado do chatbot
        ["Estou bem, obrigado por perguntar! E eu ?"]
    ],
    [
        r"(.*) você gosta de (.*)",  # Pergunta sobre os gostos
do chatbot
        ["Não sei se gosto de %2, mas e você ?"]
    ],
    [
        r"sair",  # Comando para encerrar a conversa
        ["Tchau!  Foi  um  prazer  conversar  com  você.  Até  a
próxima!"]
    ],
    [
        r"(.*)",    #  Resposta  genérica  para  entradas  não
reconhecidas
        ["Desculpe, não entendi. Pode reformular a pergunta?"]
    ]
]

# Função para aplicar reflections apenas nas respostas
def aplicar_reflections(texto, reflections):
    palavras = texto.lower().split()
    resposta_transformada = [reflections.get(palavra, palavra)
for palavra in palavras]
    return " ".join(resposta_transformada)

# Criando o chatbot
chatbot                    =                    Chat(pares,
reflections_personalizados_somente_nas_respostas_do_chat)

# Testando o chatbot
print("Chatbot: Olá! Sou um chatbot. Diga 'sair' para encerrar
a conversa.")
while True:
    entrada = input("Você: ")
    if entrada.lower() == "sair":
        print("Chatbot:  Tchau!  Foi  um  prazer  conversar  com
você.")
        break

    # Obtém a resposta sem modificar a entrada
    resposta = chatbot.respond(entrada)

    # Aplica reflections na resposta, não na entrada
    if resposta:
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
        resposta_transformada = aplicar_reflections(resposta,
reflections_personalizados_somente_nas_respostas_do_chat)
        print(f"Chatbot: {resposta_transformada}")
    else:
        print("Chatbot: Desculpe, não entendi sua pergunta.")
```

Código-fonte 1 - Exemplo de chatbot utilizando Python
Fonte: Elaborado pelo autor (2024)

Este código cria um chatbot básico utilizando a biblioteca NLTK em Python, com personalização no comportamento das respostas através do dicionário de *reflections_personalizados_somente_nas_respostas_do_chat.*

O módulo “Chat” permite criar chatbots com base em padrões de entrada e resposta.

O dicionário que está no início do código substitui palavras da resposta para torná-la mais fluida e natural. Por exemplo, se o chatbot responde com "meu carro é azul", o reflection muda o pronome para "seu carro é azul", indicando que o carro é seu (usuário). Isso é um exemplo de como adaptar apenas as respostas do chatbot e melhorar a experiência do diálogo.

Os pares contêm expressões regulares, por exemplo: "meu nome é (.*)" captura o nome informado pelo usuário e substitui no marcador %1. Já as respostas, são personalizadas para cada padrão. Exemplo: para "meu nome é Ana", a resposta seria "Olá, Ana! Como posso ajudar hoje?".

A função *aplicar_reflections(texto, reflections)* processa cada palavra da resposta do chatbot, substituindo-a pelo correspondente no dicionário “reflections”, que, nesse caso, são os pronomes pessoais “você”, “eu” e os possessivos “meu”, “seu” etc. Exemplo:

Entrada: “meu carro é azul”

Saída após reflection: “seu carro é azul”

No comando a seguir, tem-se a criação do chatbot:

chatbot = Chat(pares, reflections_personalizados_somente_nas_respostas_do_chat)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O chatbot é criado utilizando os pares definidos e o dicionário de reflections personalizados. Ele faz uso de padrões para identificar a intenção do usuário, e o dicionário de reflections para ajustar as respostas dinamicamente.

No loop principal do while(true), tem-se a entrada do usuário, a captura da mensagem digitada, o encerramento do programa se a entrada for “sair” e a resposta do Chatbot com o *respond(entrada)* encontrando a melhor correspondência nos pares. Se não há correspondência, retorna uma resposta genérica e caso algum pronome não esteja previsto no dicionário, a frase resultante permanece inalterada.

Em outras palavras, na aplicação de “reflections”, ajusta-se a resposta do chatbot com base no dicionário de reflections personalizados. A saída é exibida com substituições automáticas de pronomes para uma iteração mais natural. O programa termina quando o usuário digita “sair”.

Entretanto, todo chatbot possui limitações. Algumas delas nesse exemplo:

- O chatbot não utiliza processamento de linguagem natural avançado, dependendo apenas de padrões pré-definidos.
- Possui falta de contexto, isto é, não consegue lembrar de interações anteriores ou manter um contexto durante o diálogo.
- Possui restrições dos pares, ou seja, se o usuário digitar algo que não corresponda exatamente a um padrão definido, o chatbot dará uma resposta genérica.

As possíveis melhorias seriam melhorar a diversidade de respostas, adicionando mais pares e respostas mais elaboradas ou usar o PNL avançado, integrando modelos de aprendizado de máquina, como GPT, para respostas mais naturais. Além disso, outras melhorias seriam possíveis: adicionar um contexto, implementando lógica para armazenar e utilizar o histórico de interações do usuário, além de adicionar uma interface, substituindo o terminal por uma interface gráfica ou integrá-lo a plataformas, como Telegram ou WhatsApp, usando o Twilio.

Dica: com o Twilio, Google Colab e o seu número de WhatsApp, você pode criar um chatbot ou automatizar mensagens. Ao abrir uma conta no Twilio, você vai encontrar um passo a passo de como subir esse serviço Web.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.4 Exemplo de Agente Simples

Imagine um aspirador de pó automático que aspira e varre o chão de quatro cômodos de uma casa. Sua função é interagir com o(s) ambiente(s) e limpar todas as áreas sujas.

Figura 3 - Robô aspirador automático como exemplo de agente
Fonte: Dall-E (2024)

Seu código seria basicamente o que está apresentado a seguir. Removemos os códigos responsáveis pela leitura e tratamento dos sensores para focar no entendimento apenas do agente simples.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
class Ambiente:
    def __init__(self):
        # Define o ambiente como uma matriz 2x2 (4 cômodos)
        self.casas = [
            ["sujo", "limpo"],
            ["sujo", "sujo"]
        ]

    def exibir_ambiente(self):
        print("Estado atual do ambiente:")
        for linha in self.casas:
            print(linha)

class Agente:
    def __init__(self, ambiente):
        self.ambiente = ambiente
       self.posicao = [0, 0]  # Começa no canto superior esquerdo
(0,0)

    def perceber(self):
        # Observa o estado da casa atual
        x, y = self.posicao
        return self.ambiente.casas[x][y]

    def decidir_acao(self, estado):
        # Decide a ação com base no estado percebido
        if estado == "sujo":
            return "limpar"
        return "mover"

    def agir(self, acao):
        # Executa a ação decidida
        if acao == "limpar":
            x, y = self.posicao
            self.ambiente.casas[x][y] = "limpo"
            print(f"Limpando a posição {self.posicao}")
        elif acao == "mover":
            self.mover()

    def mover(self):
         # Move o agente para a próxima posição (simplesmente
para a direita ou para baixo)
        x, y = self.posicao
        if y < len(self.ambiente.casas[0]) - 1:  # Move para a
direita, se possível
            self.posicao = [x, y + 1]
         elif x < len(self.ambiente.casas) - 1:  # Move para
baixo, se possível
            self.posicao = [x + 1, 0]
        else:
            print("Fim do ambiente. Nada mais a fazer.")
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                self.posicao = None  # Agente para ao final do
ambiente
        print(f"Movendo para {self.posicao}")

    def executar(self):
        # Loop principal do agente
        while self.posicao is not None:
            estado = self.perceber()
            acao = self.decidir_acao(estado)
            self.agir(acao)
            self.ambiente.exibir_ambiente()

# Criando o ambiente e o agente
ambiente = Ambiente()
agente = Agente(ambiente)

# Exibindo o estado inicial do ambiente
ambiente.exibir_ambiente()

# Executando o agente
agente.executar()
```

Código-fonte 2 - Exemplo de agente
Fonte: Elaborado pelo autor (2024)

Portanto, apenas reforçando, o objetivo principal do agente simples (agente é aquele age, que pratica uma ação, isto é, um robô aspirador) é garantir que todas as posições do ambiente estejam limpas. Ele percorre o ambiente, detecta se cada posição está suja ou limpa por meio de sensores que identificam pó ao aspirar o chão, toma a decisão apropriada e executa a ação correspondente. Quando todo o ambiente estiver limpo, o agente para.

No código, a casa está representada por uma matriz 2x2. Por que uma matriz? Para que didaticamente possamos organizar os quatro ambientes hipotéticos da casa:

1) Sala de estar (elemento 1 da matriz).
2) Sala de TV (elemento 2 da matriz).
3) Corredor (elemento 3).
4) Cozinha (elemento 4).

Cada posição da matriz pode estar em dois estados: “sujo” ou “limpo”, ou seja, a sala de estar pode estar suja ou limpa, a sala de TV pode de estar suja ou limpa, e

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

assim por diante para o corredor e para a cozinha. Vamos iniciar o nosso entendimento do algoritmo da seguinte maneira:

- Sala de estar (suja).
- Sala de TV (limpa).
- Corredor (sujo).
- Cozinha (suja).

No código, iniciaríamos assim:

```
['sujo', 'limpo']
['sujo', 'sujo']
```

Código-fonte 3 - Parte do código do agente
Fonte: Elaborado pelo autor (2024)

Claro que, na prática, o robô consideraria inicialmente, tudo sujo, a não ser que o usuário altere, via aplicativo do robô, a sua configuração inicial para ele não limpar a sala de TV porque ela já está limpa, de acordo com o código. E para isso, o aplicativo já precisaria ter um mapa da casa que o próprio usuário insere no robô ao utilizá-lo pela primeira vez.

O agente tem três responsabilidades principais:

1) Perceber o ambiente, onde ele observa o estado da posição atual. Por exemplo, se o agente está na posição [0, 0] e ela está “sujo”, ele “percebe” (com base nos sensores) que precisa limpar.
2) Tomar decisões, onde após perceber o estado da posição atual, o agente decide o que fazer. Se a posição está “sujo”, ele decide “limpar”. Se a posição está “limpo”, ele decide “mover” para a próxima posição.
3) Agir no ambiente, onde ele realiza a ação decidida. Se a ação for “limpar”, ele atualiza o estado da posição para “limpo”. Se a ação for “mover”, ele se desloca para a próxima posição no ambiente.

Portanto, o passo a passo do código-fonte fica assim:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1) Executa a inicialização, onde o ambiente é configurado com o estado inicial (algumas posições sujas e outras limpas). O agente começa na posição [0, 0].
2) Executa o ciclo de percepção, decisão e ação e, enquanto o agente não percorreu todas as posições do ambiente, ele percebe e observa o estado da posição atual, decide ou escolhe entre “limpar” ou “mover”, age ou realiza a ação decidida, atualizando o estado do ambiente ou mudando de posição.
3) O término só ocorre quando o agente percorre todas as posições, e então ele para e informa que o ambiente está limpo.

Se você executar esse código no Colab, terá algo assim:

1) Para a posição [0, 0], percepção: “sujo”. Ação: Limpar → O estado da posição é atualizado para “limpo”.

```
['sujo', 'limpo']
['sujo', 'sujo']
```

Código-fonte 4 - Condição inicial dos ambientes no código
Fonte: Elaborado pelo autor (2024)

No ambiente após ação, temos o seguinte. Note a atualização do elemento [0, 0]:

```
['limpo', 'limpo']
['sujo', 'sujo']
```

Código-fonte 5 - Condição após a primeira iteração
Fonte: Elaborado pelo autor (2024)

2) Posição [0, 1], percepção: “limpo”. Ação: Mover → O agente se move para a próxima posição [1, 0]. Embora tenha uma segunda iteração, nada mudou.

3) Posição [1, 0], percepção: “sujo”. Ação: Limpar → O estado da posição é atualizado para “limpo”. Ambiente após ação (note o elemento [1, 0]):

```
['limpo', 'limpo']
['limpo', 'sujo']
```

Código-fonte 6 - Condição após a terceira iteração
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4) Posição [1, 1], percepção: “sujo”. Ação: Limpar → O estado da posição é atualizado para “limpo”. Ambiente após ação:

```
['limpo', 'limpo']
['limpo', 'limpo']
```

Código-fonte 7 - Condição após a quarta iteração
Fonte: Elaborado pelo autor (2024)

5) Fim do ambiente, então o agente percebe que todas as posições foram visitadas. Temos o término da execução.

Este agente é simples pois não mantém histórico e a ideia, neste primeiro momento, é entender o conceito de agentes de IA com um exemplo simples. Isso inclui de só considerar o estado atual da posição onde está, sem “lembrar” do que já limpou, a não ser que um aplicativo o informe.

Por isso, nosso agente simples possui uma regra simples: se está sujo → limpa, se está limpo → move. Além disso, o ambiente é fixo, visto que só funciona para matrizes pequenas e com formato conhecido. Vale mencionar que a estratégia é inflexível, pois ele sempre percorre o ambiente da mesma forma, sem planejar rotas eficientes. E não possui um planejamento global, já que não considera o ambiente como um todo para decidir ações mais otimizadas. Tudo isso que está sendo dito é para você ter uma ideia de quantas nuances um simples robô que varre o chão teria que ter para ser um bom robô.

Portanto, embora simples, este modelo é útil para entender a estrutura e o comportamento de agentes inteligentes, sendo um ponto de partida para desenvolver agentes mais complexos.

1.5 Aplicações dos Agentes

Os agentes de IA possuem aplicações diversificadas em diferentes áreas. Na área da saúde, esses agentes atuam como assistentes virtuais para pacientes, fornecendo informações sobre sintomas, agendando consultas e enviando lembretes sobre medicamentos. Eles também são usados no auxílio de diagnósticos automatizados, analisando imagens médicas para detectar doenças como câncer e

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

condições cardiovasculares, além de monitorar pacientes em tempo real, enviando alertas em caso de anomalias, especialmente em unidades de terapia intensiva.

Na indústria, os agentes de IA desempenham papéis importantes, como em sistemas de manutenção preditiva, que monitoram equipamentos e identificam padrões que indicam possíveis falhas, reduzindo o tempo de inatividade e prevendo manutenções. Eles também são utilizados em processos de controle de qualidade, inspecionando produtos por meio de visão computacional para identificar defeitos, itens fora de conformidade e formatos, e na otimização da logística, coordenando a alocação de recursos e melhorando a eficiência das cadeias de suprimentos.

No comércio e varejo, os agentes inteligentes personalizam experiências de consumo por meio de assistentes de recomendação que analisam dados de compra e navegação para sugerir produtos relevantes. Além disso, agentes integrados aos sistemas de atendimento ao cliente (por exemplo: WhatsApp) fornecem suporte 24 horas, enquanto soluções de gestão de inventário monitoram níveis de estoque em tempo real e automatizam o reabastecimento conforme necessário.

Na educação, agentes de IA oferecem tutoria personalizada, adaptando conteúdos e metodologias ao ritmo de aprendizado de cada aluno. Eles também analisam o desempenho dos estudantes e sugerem melhorias, além de atuar como assistentes de aprendizado, respondendo dúvidas específicas e auxiliando no acesso a materiais de estudo.

Na agricultura, esses agentes monitoram cultivos por meio de sensores e imagens provenientes de drones, identificando pragas, doenças e necessidades de irrigação através das cores das lavouras (folhas verdes indicam plantas saudáveis, enquanto outras cores podem indicar variações dessa saúde). Eles também otimizam a gestão de recursos hídricos com base em dados meteorológicos e do solo, e realizam previsões de safras analisando históricos climáticos e padrões de plantio.

No setor de transporte e mobilidade, os agentes de IA são importantes para o funcionamento de veículos autônomos, processando dados de sensores para tomar decisões em tempo real, como direção e frenagem. Eles também são usados em sistemas de gestão de tráfego, monitorando o fluxo de veículos e otimizando sinais para reduzir congestionamentos, além de assistentes de navegação que sugerem rotas mais rápidas ou econômicas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em finanças, os agentes atuam como assistentes financeiros, ajudando usuários a gerenciar suas finanças pessoais e oferecendo dicas de economia. Eles também detectam fraudes, analisando transações em tempo real para identificar padrões suspeitos, e auxiliam na análise de mercado, processando grandes volumes de dados para prever tendências e apoiar decisões de investimento.

No entretenimento, os agentes de IA geram conteúdo personalizado, criando músicas, roteiros ou vídeos com base nas preferências dos usuários. Eles são usados em plataformas de streaming para recomendar conteúdos relevantes e em jogos, fornecendo experiências adaptativas para os jogadores, como inimigos controlados por IA que ajustam o desafio com base no desempenho do jogador.

Por fim, na segurança patrimonial, os agentes monitoram vídeos de câmeras de vigilância em tempo real, detectando comportamentos suspeitos e prevenindo incidentes. Eles também analisam comportamentos e ameaças cibernéticas, identificando padrões anômalos em redes para alertar sobre possíveis ataques, e reforçam a autenticação em sistemas críticos por meio de biometria e padrões comportamentais. Esses exemplos mostram como os agentes de IA estão moldando diversos setores, oferecendo soluções inovadoras e aumentando a eficiência em processos e serviços.

Apenas como reforço do que foi dito antes, seja qual for a aplicação do agente, no *backend* é uma Machine Learning ou uma rede neural, a depender do nível de complexidade dessa aplicação.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 CLASSIFICAÇÃO DE AGENTES

Neste primeiro momento, a classificação de agentes em sistema de IA é realizada com base na forma como eles percebem o ambiente, isto é, na forma como eles interagem com o ambiente, e como tomam as decisões e como agem para alcançar seus objetivos. Já no segundo momento, os agentes podem ser categorizados de acordo com sua complexidade e capacidade de adaptação.

2.1 Agentes Reflexivos Simples

Os **agentes reflexivos simples** são os mais básicos entre todos. Eles tomam decisões exclusivamente com base no estado atual do ambiente, sem considerar nenhum histórico ou consequências futuras. É o exemplo do agente robô varredor do Colab anterior.

Eles funcionam utilizando regras de condição-ação, como: “se a temperatura for baixa, ligue o aquecedor”. Por exemplo, um termostato que liga o aquecedor quando a temperatura cai abaixo de um limite predefinido é um agente reflexivo simples.

Apesar de serem extremamente eficientes em ambientes previsíveis, sua principal limitação é a incapacidade de lidar com situações complexas ou ambientes onde o estado atual não fornece informações completas. Por isso que um agente reflexivo simples pode ser confundido com um chatbot simples, já que o chatbot simples também possui limitações claras em relação à sua funcionalidade. O chatbot funciona apenas com base em regras simples de condição-ação, ou seja, responde a entradas específicas do usuário com respostas predefinidas, sem considerar o contexto ou histórico da conversa.

2.2 Agentes Baseados em Modelo

Já os **agentes baseados em modelo** são uma evolução dos reflexivos simples, pois mantêm um modelo interno que representa o estado do ambiente.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Esse modelo permite que o agente insira informações sobre o ambiente mesmo quando ele não está completamente observável.

Por exemplo, um robô que navega em um ambiente parcialmente mapeado utiliza um modelo interno para estimar sua posição e prever o impacto de suas ações.

Esses agentes conseguem funcionar em ambientes mais dinâmicos e incertos, mas sua implementação requer mais memória e maior capacidade computacional, pois ele vem carregado com um modelo nativo e preparado para atuar naquele ambiente em específico.

Um exemplo clássico de um **agente baseado em modelo** é um robô de navegação autônomo, como um aspirador de pó robótico (exemplo mais avançado do que o reflexivo simples) que opera em um ambiente parcialmente observável.

O robô aspirador de pó que mantém um mapa interno do ambiente que representa o layout do espaço, como paredes, móveis e áreas limpas ou sujas, é um exemplo que possui modelo. Esse modelo é atualizado à medida que o robô explora ou recebe novas informações do ambiente.

Para esse caso do aspirador, o robô utiliza sensores (como ultrassom, LIDAR, infravermelhos simples, câmeras ou encoders nas rodinhas que memorizam os passos já dados) para perceber obstáculos, identificar sujeira ou reconhecer áreas já limpas. Ele não consegue observar todo o ambiente ao mesmo tempo, mas pode inferir estados não observáveis com base no modelo interno.

Como o robô possui um modelo, sua decisão será baseada nesse modelo. Geralmente, quanto mais caro e famosa for a marca do robô, teoricamente será a sua atuação na limpeza da casa, pois houve um investimento maior em recursos computacionais nele.

Com o modelo interno, o robô pode decidir aonde ir em seguida, mesmo que não tenha visibilidade direta de todas as áreas. Por exemplo, se o sensor indica que a área à frente está limpa, ele verifica o modelo interno para escolher outro local ainda sujo. Ou, se um obstáculo bloqueia o caminho, ele usa o modelo interno para planejar uma rota alternativa.

Vamos considerar outro exemplo de cenário, imagine que o robô está em uma sala parcialmente mapeada e encontra um obstáculo inesperado (uma cadeira fora do

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

lugar). Ele atualiza seu modelo interno para registrar a cadeira e calcular um caminho alternativo para continuar limpando. Ele registra cruzando a informação do encoder em suas rodinhas com uma foto, por exemplo.

Na área da robótica, um encoder é um dispositivo sensor que mede o movimento, a posição ou a velocidade de componentes mecânicos, como motores ou eixos. Ele converte informações físicas de movimento (como rotação ou deslocamento linear) em sinais elétricos que podem ser interpretados por sistemas de controle, permitindo que o robô monitore, memorize e ajuste seus movimentos com precisão.

Você pode estar pensando: por que este robô é baseado em modelo? Porque ele não depende apenas de regras simples como “se está sujo, limpe” como vimos aqui antes. Ele mantém um estado interno (modelo) que descreve o ambiente e pode inferir informações sobre áreas não observadas diretamente. E ele pode planejar ações com base nesse modelo interno para alcançar seu objetivo, mesmo em ambientes dinâmicos ou parcialmente observáveis.

E assim, o agente baseado em modelo, tem a capacidade de navegar em ambientes complexos, como ambientes grandes ou com obstáculos inesperados. Ele evita áreas já limpas, otimizando suas ações. E mesmo que o sensor não detecte todas as áreas de uma vez, o modelo ajuda a inferir informações ausentes.

Em aplicações reais, temos: drones, carros autônomos e jogos de computadores, onde os personagens não jogáveis, os famosos NPC (Non-Playable-Character) são personagens que fazem parte do jogo digital, mas não são controlados pelo jogador, porém interagem com o jogador dando dicas, orientações, fornecendo informações, missões ou itens no jogo. Um NPC famoso é o Toad, do Super Mario Bros. Esse personagem que auxilia o Mário (jogador) em sua jornada é um clássico agente baseado em modelo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

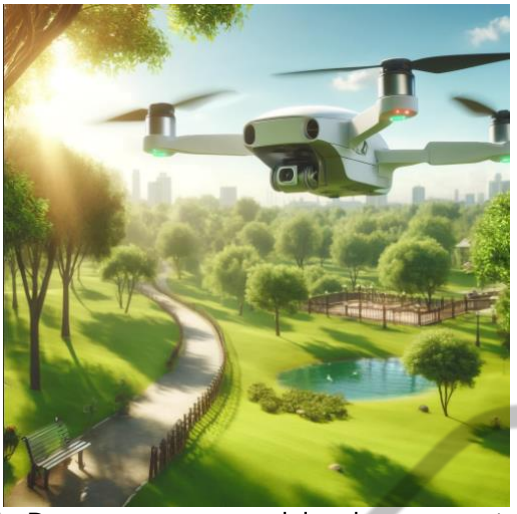

Figura 4 - Drones possuem modelos de voos e estabilidade
Fonte: Dall-E (2024)

Figura 5 - Carros autônomos da Tesla possuem modelos com regras de trânsito
Fonte: Dall-E (2024)

Figura 6 - Toad, da saga Super Mario Bros
Fonte: Mario Wonder World Wiki (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2.3 Agentes Orientados a Metas

Os agentes orientados a metas **adicionam um elemento de planejamento** às decisões, isto é, uma meta. Eles não apenas observam o ambiente e reagem, mas também avaliam as ações com base em metas específicas que precisam alcançar.

Por exemplo, um aplicativo de GPS (Waze ou Google Maps) calcula a melhor rota para um destino com base na posição atual e no objetivo final. Isso permite que o agente tome decisões direcionadas e adaptadas à situação, mas depende de uma definição clara de objetivos e metas.

Portanto, agentes baseados em metas são sistemas de IA projetados para atuar com base em objetivos específicos. Esses agentes não se limitam a reagir ao ambiente como os agentes reflexivos; em vez disso, eles planejam suas ações para alcançar estados finais desejados, conhecidos como “metas”. Esse planejamento é orientado por uma análise de como cada ação contribui para aproximar o agente da meta. Eles utilizam um modelo do ambiente para avaliar as consequências de suas ações e determinar a melhor sequência de passos.

O funcionamento de um agente baseado em metas começa com a percepção do ambiente. Ele observa seu estado atual por meio de sensores ou informações externas e compara esse estado com a meta desejada (nessa altura, você deve estar pensando naquele exato momento que o Waze calcula e apresenta a rota para você seguir o seu caminho).

A cada instante, caso perceba que ainda não atingiu a meta (isto é, não chegou no destino, ou o congestionamento está piorando, ou você teve que fazer um desvio inesperado), o agente replaneja as ações necessárias para reduzir a diferença entre o estado atual e o estado final desejado, isto é, ele calcula ou recalcula a melhor rota utilizando algoritmos de planejamento, como Dijkstra (Kang, 2008).

Portanto, esse planejamento considera possíveis mudanças no ambiente, permitindo que o agente adapte seu comportamento se surgir um obstáculo ou nova informação.

Um carro autônomo também pode se enquadrar nesse tipo de agente e no algoritmo de Dijkstra (Qing, 2017).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Um outro exemplo seria um robô em um armazém com a meta de mover uma caixa do ponto A para o ponto B. O agente, inicialmente, observa que a caixa está no ponto A e que o ponto B ainda não foi alcançado. Ele planeja pegar a caixa, mover-se até o ponto B e colocá-la no chão. Após cada ação, verifica o progresso em direção à meta e ajusta suas decisões se necessário. Ao final do processo, o agente garante que a meta foi cumprida, demonstrando sua capacidade de raciocínio orientado a alcançar objetivos.

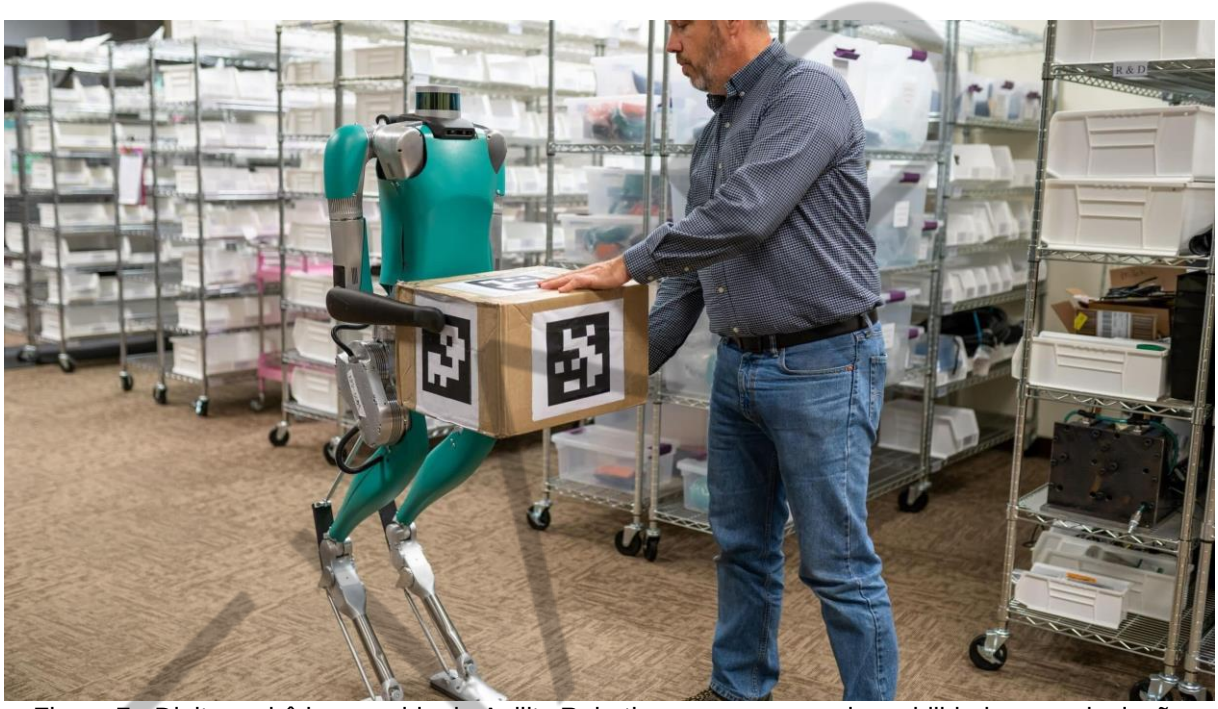

Figura 7 - Digit, o robô humanoide da Agility Robotics com recursos de mobilidade e manipulação
Fonte: Uol (2020)

Portanto, os agentes baseados em metas têm várias vantagens. Eles tomam decisões direcionadas a objetivos específicos, o que evita ações desnecessárias e aumenta a eficiência.

No entanto, esses agentes apresentam limitações, como a necessidade de metas bem definidas e custos computacionais elevados para planejar ações em ambientes amplos ou altamente dinâmicos.

O diferencial dos agentes baseados em metas em relação a outros tipos é o foco em alcançar estados finais específicos. Enquanto os agentes reflexivos reagem diretamente ao ambiente, os agentes baseados em metas priorizam alcançar o

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

objetivo, mesmo que o ambiente mude ao longo do processo ou que você tenha que passar em uma estrada de terra esburacada para chegar mais rápido no seu destino.

A seguir, veremos os agentes baseados em utilidade que avaliam a qualidade entre várias opções.

2.4 Agentes Baseados em Utilidade

Os agentes baseados em utilidade representam uma evolução significativa em relação aos agentes orientados a metas, pois não se limitam a alcançar um objetivo predefinido.

Em vez disso, eles introduzem uma camada adicional de sofisticação ao avaliar a qualidade de diferentes ações e resultados, utilizando uma função de utilidade para determinar o “valor” de cada decisão possível. Essa abordagem permite que esses agentes tomem decisões mais flexíveis e otimizadas, especialmente em cenários onde existam múltiplos objetivos ou prioridades que precisam ser equilibrados.

A principal característica de um agente baseado em utilidade é sua capacidade de lidar com a complexidade de ambientes onde as escolhas nem sempre são binárias, mas envolvem compensações entre fatores.

Por exemplo, em um veículo autônomo, o agente não apenas decide como chegar ao destino, mas também considera aspectos como conforto, tipo de estrada, segurança e eficiência energética do carro. Ele avalia cada possível trajetória com base em uma função de utilidade que atribui um peso ou prioridade a cada critério.

Dessa forma, o veículo pode escolher a melhor rota que equilibre um tempo de viagem mais curto, a segurança de evitar áreas perigosas e o conforto de reduzir curvas bruscas por rodovias ou ruas bem asfaltadas.

Essa flexibilidade é uma das maiores vantagens dos agentes baseados em utilidade, pois permite que eles adaptem seu comportamento em tempo real para atender a diferentes demandas e condições.

Em ambientes dinâmicos, onde as circunstâncias mudam rapidamente, como no tráfego urbano ou em emergências, a capacidade de considerar diversas opções e selecionar a melhor com base em múltiplos critérios torna esses agentes muito

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

eficazes. Essa abordagem também permite que eles lidem com incertezas no ambiente, atribuindo valores probabilísticos a diferentes resultados e escolhendo a ação que maximize o valor esperado.

No entanto, a construção e a definição de uma função de utilidade adequada representam um desafio significativo. Para que o agente opere de maneira eficaz, essa função deve capturar com precisão as prioridades e preferências do sistema ou do usuário final.

Isso pode envolver pesos conflitantes, como a priorização da segurança sobre a eficiência em um veículo autônomo, ou a necessidade de equilibrar custos e benefícios em sistemas de logística. A criação de uma função de utilidade que reflita essas prioridades de forma realista requer um entendimento profundo do contexto operacional do agente e pode demandar modelagem complexa.

Embora a maior flexibilidade e capacidade de tomada de decisão tornem os agentes baseados em utilidade uma escolha ideal para muitos problemas, sua implementação é computacionalmente mais intensiva.

Avaliar todas as possíveis ações e calcular sua utilidade, especialmente em sistemas complexos com inúmeras variáveis, pode exigir recursos significativos. Apesar dessas dificuldades, a abordagem baseada em utilidade é amplamente utilizada em diversas áreas, como robótica avançada, veículos autônomos, sistemas de recomendação e planejamento estratégico.

Graças ao excelente nível computacional que os celulares apresentam hoje, o Waze e Google Maps estão sempre se atualizando. Porém, celulares mais simples rodam modelos mais simples, enquanto que os modelos mais sofisticados conseguem entregar uma maior experiência na condução do motorista.

2.5 Agentes de Aprendizado

Os **agentes de aprendizado** representam uma das formas mais avançadas de agentes inteligentes, visto que são capazes de ajustar e melhorar seu desempenho com base na experiência acumulada.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Ao contrário de outros tipos de agentes que dependem de regras ou modelos predefinidos, os agentes de aprendizado possuem a capacidade de modificar seu comportamento conforme interagem com o ambiente.

Essa adaptabilidade é o que os torna particularmente eficazes em ambientes dinâmicos e imprevisíveis, onde é difícil prever todas as condições possíveis de operação antecipadamente.

O aprendizado desses agentes ocorre por meio da análise de dados coletados ao longo de suas interações. A partir desses dados, eles identificam padrões e ajustam suas estratégias para maximizar o sucesso de suas ações.

Isso é possível graças às técnicas de Machine Learning, como aprendizado supervisionado, aprendizado não supervisionado e aprendizado por reforço.

No **aprendizado supervisionado**, o agente utiliza dados rotulados, ou seja, entradas e saídas esperadas, para ajustar seu modelo e prever corretamente novos dados.

O **aprendizado não supervisionado**, quando aplicado a agentes de aprendizado, é um método para ensinar esses agentes a identificar padrões ou estruturas nos dados sem depender de rótulos ou respostas predefinidas. Nesse contexto, o agente não recebe informações explícitas sobre o que deve aprender ou quais são os resultados esperados. Em vez disso, ele analisa os dados (que devem ser abundantes) por conta própria, buscando padrões subjacentes, agrupamentos ou representações úteis para tomar decisões futuras.

Já no **aprendizado por reforço**, o agente aprende por tentativa e erro, recebendo recompensas ou penalidades com base nas ações realizadas, o que o incentiva a explorar novas estratégias e refinar suas decisões para alcançar melhores resultados.

Um exemplo clássico de agente de aprendizado é um chatbot moderno. Esses agentes começam com um conjunto básico de respostas e, ao longo do tempo, aprendem a interpretar melhor as intenções dos usuários e a oferecer respostas mais precisas e relevantes.

Por exemplo, ao interagir com usuários reais, o chatbot pode coletar feedback sobre suas respostas, ajustando seu comportamento para se alinhar melhor às

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

necessidades e expectativas dos interlocutores. Além disso, ao utilizar técnicas como aprendizado por reforço, o chatbot pode experimentar diferentes formas de interação e descobrir quais estratégias resultam em maior satisfação do usuário.

Uma maneira de você dar o reforço é clicando no joinha a cada resposta do ChatGPT, por exemplo.

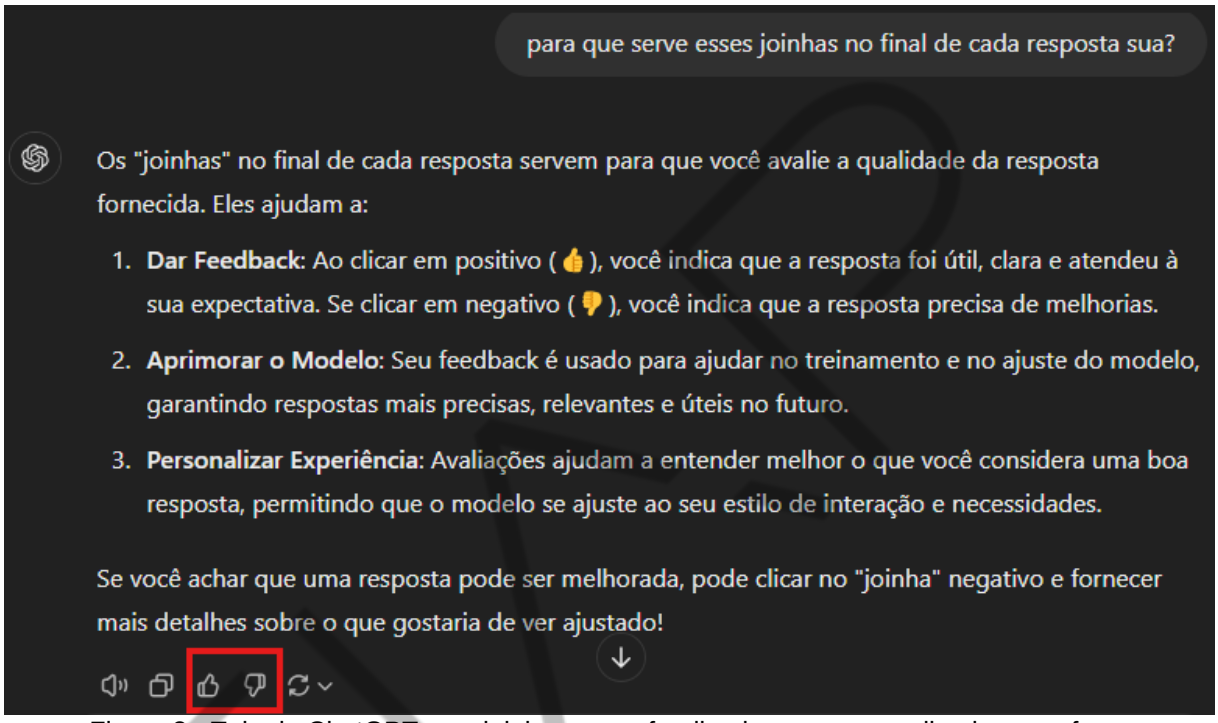

Figura 8 - Tela do ChatGPT e os joinhas como feedback para o aprendizado por reforço
Fonte: ChatGPT (2024)

Embora a capacidade de aprendizado torne esses agentes altamente adaptáveis e eficazes, ainda existem desafios associados à sua implementação. Um dos fatores críticos para o sucesso de agentes de aprendizado é a qualidade e a quantidade dos dados disponíveis para treinamento.

Dados insuficientes ou enviesados podem levar a modelos que falham em generalizar ou que apresentam resultados inconsistentes em novos cenários. Além disso, a complexidade computacional envolvida no treinamento desses agentes, especialmente em sistemas baseados em grandes conjuntos de dados ou em ambientes com muitas variáveis, pode ser significativa.

Ainda assim, a capacidade de aprendizado desses agentes permite que eles se destaquem em aplicações onde a adaptabilidade é essencial. Em áreas como robótica, diagnóstico médico, sistemas de recomendação e análise preditiva, os

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

agentes de aprendizado oferecem uma abordagem poderosa para resolver problemas complexos.

2.6 Multiagentes

Os **sistemas multiagentes** são uma abordagem mais avançada dentro da IA, na qual diversos agentes autônomos operam em um ambiente compartilhado.

Esses agentes podem trabalhar em conjunto, competir entre si ou combinar ambos os comportamentos para alcançar objetivos individuais ou coletivos.

Você sabia que pode fazer um agente para testar o seu agente?

A essência desses sistemas é a capacidade dos agentes de interagir, seja para compartilhar informações ou para tomar decisões que impactem tanto seus próprios interesses quanto os de outros agentes no sistema.

Essa interação torna os sistemas multiagentes particularmente úteis para resolver problemas complexos e altamente escaláveis, como logística, simulação de tráfego e mercados financeiros.

A cooperação entre agentes é uma das formas mais comuns de interação em sistemas multiagentes. Nesse caso, os agentes trabalham juntos para atingir um objetivo comum, coordenando suas ações para maximizar a eficiência global.

Um exemplo prático disso ocorre em robótica colaborativa, como em armazéns automatizados. Robôs colaborativos podem dividir tarefas de organização e transporte de itens, evitando colisões e otimizando o tempo de execução. A comunicação entre eles garante que cada robô saiba qual tarefa deve executar e onde estão os outros agentes, permitindo que o sistema funcione como uma unidade integrada e eficiente.

Essa é uma ideia presente nos filmes futuristas para um trânsito quase perfeito: cada carro atua como um agente, mas no trânsito, ele se torna um multiagente, participando de cada decisão do veículo vizinho e, assim, evitando qualquer tipo de colisão.

Outro exemplo, mais complexo e atual, é o uso de sistemas multiagentes em um chatbot que utiliza PLN para buscar respostas em milhares de arquivos PDF para

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

alimentar um canal de atendimento, como o WhatsApp. Você pode fazer um agente que lide com dados financeiros, outro que lide que dados jurídicos, outro que lide com agendamentos, e assim por diante.

Esse modelo combina a força de diferentes agentes especializados para lidar com várias tarefas, como interpretar a pergunta do usuário, buscar informações em grandes volumes de dados, atualizar periodicamente o banco de respostas e garantir que a interação seja eficiente. Cada agente desempenha um papel específico e colabora ou opera de forma independente para atingir o objetivo de fornecer respostas precisas e atualizadas.

Por exemplo, seguindo com esse exemplo de canal de atendimento ao usuário, um **agente de interpretação** seria responsável por analisar a pergunta do usuário usando técnicas de PLN. Ele identificaria a intenção por trás da pergunta e extrairia palavras-chave ou conceitos importantes, que serão utilizados para buscar a resposta correta. Esse agente também poderia reformular a pergunta, caso necessário, para garantir que ela esteja otimizada para consulta nos documentos disponíveis.

Em seguida, um segundo **agente de busca** entraria em ação. Esse agente seria responsável por navegar pelos milhares de arquivos PDF armazenados em um banco de dados ou sistema de arquivos distribuído. Ele usaria técnicas de indexação, como a construção de índices invertidos ou tokenização (vetorização) de texto, para localizar rapidamente os trechos mais relevantes.

Ferramentas como Elasticsearch ou bibliotecas como PyPDF podem ser integradas para facilitar essa tarefa. Em um sistema multiagente, o trabalho de busca pode ser distribuído entre vários agentes, cada um responsável por um subconjunto dos arquivos ou uma área temática específica. Essa distribuição permite que a consulta seja realizada de forma paralela, reduzindo o tempo necessário para encontrar uma resposta.

Outro agente importante (e paralelo) seria o **agente de atualização**, que estaria responsável por monitorar periodicamente os arquivos PDF e verificar se há novos documentos ou atualizações nos existentes. Esse agente garantiria que o banco de dados de conhecimento utilizado no canal de atendimento estivesse sempre atualizado, reindexando os documentos quando necessário. Ele também poderia

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

identificar padrões de modificação nos documentos, priorizando aqueles que sofrem alterações frequentes.

Após a busca, um terceiro **agente de análise e validação** consolidaria as informações retornadas pelos agentes de busca. Ele compararia os trechos mais relevantes encontrados nos arquivos com a intenção original da pergunta, classificando as respostas com base em sua relevância e confiabilidade. Esse agente poderia utilizar modelos de aprendizado de máquina ou técnicas de similaridade de texto para validar a precisão da resposta.

Finalmente, o quarto **agente de comunicação** seria responsável por formatar e entregar a resposta ao usuário. Ele garantiria que a resposta fosse clara, bem estruturada e, se necessário, acompanhada de links para os documentos originais. Caso a resposta fosse inconclusiva ou ambígua, esse agente poderia solicitar ao usuário mais informações ou direcionar a pergunta para um humano.

Esse sistema multiagente não apenas garante eficiência no processamento e na entrega das respostas, mas também é escalável e resiliente. Por exemplo, se o número de documentos aumentar significativamente, novos agentes de busca podem ser adicionados para dividir a carga. Além disso, a modularidade do sistema permite que agentes especializados sejam aprimorados individualmente sem afetar o restante do chatbot.

Uma aplicação real desse exemplo que acabamos de descrever seria um chatbot (moderno, claro) jurídico que responde a perguntas com base em milhares de PDFs de legislação, de jurisprudência e de doutrina. Nesse caso, o agente de interpretação entenderia os termos jurídicos complexos, o agente de busca navegaria pelos documentos legais, e o agente de atualização garantiria que novas leis ou decisões fossem incorporadas ao banco de dados regularmente.

Por outro lado, em um sistema multiagente, a competição é outro aspecto relevante, especialmente em cenários onde os recursos são limitados e cada agente busca maximizar seus próprios interesses.

Um exemplo disso é encontrado em agentes de negociação utilizados em mercados financeiros. Esses agentes competem entre si para realizar transações mais vantajosas, baseando-se em estratégias de otimização de preços e previsão de mercado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Nesse tipo de sistema, a interação entre os agentes pode ocasionar resultados dinâmicos e ao mesmo tempo imprevisíveis, que precisam ser validados pelo usuário, representando um equilíbrio complexo entre cooperação limitada e competição intensa.

Por fim, um dos maiores desafios dos sistemas multiagentes é a coordenação e a comunicação entre os agentes, especialmente em cenários onde o número de agentes é elevado ou o ambiente é dinâmico.

A troca de informações deve ser eficiente para evitar conflitos e redundâncias, mas também precisa ser robusta o suficiente para lidar com falhas ou atrasos na comunicação. Além disso, a coordenação exige que os agentes tomem decisões baseadas não apenas em seus próprios estados, mas também levando em conta o comportamento e as intenções dos outros agentes no sistema.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 COMPARATIVO

Resgatando de forma resumida tudo que foi discutido até aqui, cada tipo de agente possui características únicas que o torna adequado para situações específicas.

Os agentes reflexivos são ideais para tarefas simples e previsíveis, enquanto os baseados em modelo e orientados a metas são úteis em ambientes mais complexos e dinâmicos.

Agentes baseados em utilidade lidam bem com múltiplas prioridades, e os de aprendizado são adaptáveis a mudanças constantes. Por outro lado, os sistemas multiagentes expandem essas capacidades ao permitir que múltiplos agentes colaborem ou concorram em ambientes amplos e diversificados.

Essa diversidade de agentes reflete a amplitude das aplicações de IA que existem hoje, desde sistemas simples de automação até soluções sofisticadas para problemas globais.

A seguir, você encontrará um resumo comparativo sobre a necessidade de memória, o planejamento, a necessidade de avaliar as consequências dos resultados por parte do usuário, a capacidade do modelo lidar com as incertezas do ambiente e o fator de ter ou não presente alguma técnica de aprendizado de máquina.

3.1 Resumo comparativo

Tipo de agente	Memória	Planejamento	Avalia consequências	Lida com incertezas	Aprendizado
Reflexivo Simples	Não	Não	Não	Não	Não
Baseado em Modelo	Sim	Não	Sim	Sim	Não
Orientado a Metas	Sim	Sim	Sim	Sim	Não
Baseado em Utilidade	Sim	Sim	Sim	Sim	Não
Agente de Aprendizado	Sim	Sim	Sim	Sim	Sim
Sistemas Multiagentes	Depende	Depende	Depende	Sim	Depende

Quadro 1 - Comparativo entre os tipos de agentes de IA
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 O CHAT GPT POSSUI QUAIS TIPOS DE AGENTES?

O ChatGPT se enquadra no tipo de **agente baseado em aprendizado**, mais especificamente um **agente de aprendizado supervisionado e por reforço**, que também possui características de **agente baseado em utilidade**. Ele não é puramente reflexivo nem exclusivamente orientado a metas, mas combina múltiplos aspectos para oferecer interações eficazes e adaptáveis.

Por que o ChatGPT é um Agente de Aprendizado? Porque ele foi treinado utilizando técnicas de aprendizado de máquina, como o aprendizado supervisionado e o aprendizado por reforço, baseados em grandes volumes de dados textuais.

Durante seu treinamento, recebeu um conjunto de exemplos rotulados, onde os pares de entrada e saída esperados foram fornecidos (aprendizado supervisionado). Além disso, seu modelo foi refinado com Aprendizado por Reforço com Feedback Humano (RLHF), onde respostas geradas foram avaliadas e ajustadas com base no que é considerado apropriado ou útil.

Essa capacidade de aprendizado permite ao ChatGPT gerar respostas coerentes e contextualmente relevantes, mesmo quando lida com perguntas ou tarefas novas. Ele não segue regras predefinidas como agentes reflexivos; em vez disso, constrói respostas baseadas em padrões aprendidos durante o treinamento.

O ChatGPT também exibe características de um agente baseado em utilidade, pois, ao gerar uma resposta, ele busca maximizar sua função de utilidade implícita. Essa função avalia:

1) **Relevância:** onde a resposta precisa estar alinhada com a pergunta do usuário.
2) **Coerência:** onde a resposta é logicamente consistente.
3) **Clareza:** onde a resposta é compreensível para o usuário.
4) **Utilidade geral:** onde a resposta atende ao objetivo da interação.

Embora o ChatGPT não tenha uma função de utilidade explícita como sistemas projetados para problemas específicos, a arquitetura subjacente busca gerar a saída

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

que maximiza a probabilidade de ser aceita como adequada pelo usuário, com base nos dados de treinamento.

Por que não é Reflexivo Simples ou Baseado em Metas? Porque não responde a padrões fixos de entrada. Ele constrói respostas dinamicamente com base na análise do contexto e na geração de linguagem natural. Ele também não é estritamente baseado em metas, pois não trabalha com um objetivo único e predefinido, como encontrar o caminho mais curto ou concluir uma tarefa específica. Em vez disso, ele busca manter uma interação fluida e informativa.

O ChatGPT é um Sistema Multiagente? Em sua forma atual, ele não é considerado um sistema multiagente no sentido clássico, onde diferentes agentes colaboram ou competem em um ambiente compartilhado.

No entanto, ele pode ser integrado a sistemas multiagentes, como um chatbot avançado que colabora com outros agentes especializados (por exemplo, agentes de busca, agentes de validação, agentes de integração de dados) ou o uso dos prompts pré-pronto e disponíveis em diversas plataformas que utilizam o GPT como tecnologia de *backgroud*.

Ainda não se conhece de forma pública e de forma exata qual é a fórmula do ChatGPT, por isso concluímos essa seção como “traços” dos tais agentes que definimos.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

5 ESTRUTURA DE FUNCIONAMENTO DE UM AGENTE DE IA

Como reforço, vamos explorar a estrutura de funcionamento de um agente de IA e mostrar que ele se baseia em um ciclo contínuo de **percepção**, **decisão** e **ação**, que permite ao agente interagir com o ambiente de maneira eficaz e adaptável.

Figura 9 - Estrutura de funcionamento de um agente de IA
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

Esse ciclo reflete a essência de qualquer sistema de IA autônoma, pois representa como o agente percebe mudanças no ambiente, toma decisões com base nessas informações e como age para alcançar seus objetivos.

Portanto, essa abordagem modular é o núcleo da maioria dos agentes inteligentes, sejam eles simples ou avançados.

No primeiro estágio, a **percepção**, o agente coleta dados do ambiente por meio de sensores ou outros meios de entrada. Esses dados podem ser informações físicas, como temperatura, localização ou sinais visuais, ou dados abstratos, como texto, sons ou valores numéricos em um sistema digital.

Após a coleta, o agente processa os dados brutos para transformá-los em informações úteis. Esse processamento pode incluir filtragem de ruído, extração de padrões ou interpretação semântica, dependendo da natureza do agente e de seu objetivo. A qualidade da percepção é crucial, pois determina a precisão e a relevância das informações usadas nos estágios seguintes.

O segundo estágio, a **decisão**, é onde o núcleo da inteligência do agente entra em ação. Com base nas informações percebidas, o agente utiliza algoritmos para planejar e decidir suas próximas ações. Esses algoritmos podem variar de regras simples "if-else" até técnicas complexas de aprendizado de máquina, como redes neurais (Backpropagation) ou aprendizado por reforço (Deep Q-Learning — DQN).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Durante essa etapa, o agente avalia as opções disponíveis e seleciona aquela que melhor se alinha com seus objetivos ou que maximiza uma função de utilidade. Por exemplo, em um jogo, o agente pode analisar diferentes estratégias e prever os resultados de suas ações antes de escolher a melhor jogada. A eficácia dessa etapa depende tanto da capacidade do agente de raciocinar quanto da qualidade das informações percebidas.

No estágio final, **a ação**, o agente executa a decisão tomada. Isso pode envolver alterar o estado físico de um ambiente, como mover um robô para uma nova posição, ou realizar uma ação digital, como enviar uma mensagem ou acionar um processo. O sucesso da ação depende de sua implementação correta e da capacidade do agente de interagir efetivamente com o ambiente. Após agir, o agente avalia os resultados e reinicia o ciclo, adaptando suas percepções e decisões às novas condições do ambiente.

Além desse ciclo básico, a maneira como o agente representa o ambiente é fundamental para seu funcionamento. Um ambiente pode ser **totalmente observável**, onde o agente tem acesso a todas as informações relevantes em tempo real, ou **parcialmente observável**, onde o agente precisa inferir informações com base em dados limitados ou incompletos.

Por exemplo, um jogo de xadrez é um ambiente totalmente observável, enquanto a navegação de um robô em uma sala desconhecida é parcialmente observável.

O ambiente também pode ser **determinístico**, onde as ações têm resultados previsíveis, ou **estocástico**, onde há incerteza nos resultados das ações. A navegação em um mapa com regras fixas é determinística, enquanto a interação com outros agentes humanos, como em negociações, pode ser estocástica.

Essa estrutura de percepção, decisão e ação, juntamente com a forma como o ambiente é representado, forma a base para o design e a operação de agentes de IA. Esses conceitos não apenas explicam como os agentes funcionam, mas também fornecem uma estrutura para projetar sistemas mais avançados e especializados, que podem operar de forma eficiente em uma ampla variedade de ambientes e aplicações.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6 FERRAMENTAS

Nesta seção, vamos sugerir algumas ferramentas que atualmente estão em alta no mercado de desenvolvimento para soluções de agentes de IA.

6.1 Llama

Como ferramentas para construir agentes, atualmente temos o Llama (Large Language Model Meta AI), desenvolvido pela Meta.

É uma das opções emergentes para construir agentes inteligentes, especialmente aqueles que precisam interpretar texto, responder a perguntas, ou mesmo gerar linguagem natural em interações com usuários, isto é, PLN. Aplicações: criação de chatbots, assistentes virtuais e agentes de busca baseados em PLN.

Juntamente com outras ferramentas populares, como TensorFlow, PyTorch, e bibliotecas especializadas, o Llama é uma alternativa viável e competitiva no mercado atualmente.

6.2 TensorFlow

O TensorFlow é um framework de aprendizado de máquina altamente escalável e flexível, desenvolvido pelo Google. É muito utilizado para treinar redes neurais, desde simples modelos de classificação até agentes complexos de aprendizado por reforço.

Ele integra bem com outras ferramentas, como Keras, e suporta treinamento distribuído para lidar com grandes volumes de dados.

A aplicação mais comum do TensorFlow é a criação de agentes que usam aprendizado profundo, como redes neurais convolucionais para visão computacional.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.3 PyTorch

Uma alternativa ao TensorFlow, conhecida por sua flexibilidade e facilidade de uso, particularmente em pesquisa acadêmica. Oferece suporte direto para aprendizado profundo e é amplamente utilizado para treinar modelos em aprendizado supervisionado e por reforço. Entre as aplicações, temos: agentes baseados em PLN e visão computacional, com suporte a modelos dinâmicos e experimentação rápida.

6.4 OpenAI Gym

Trata-se de uma API voltada para aprendizado por reforço, oferecendo um conjunto de ambientes simulados para treinar agentes, como jogos, robótica e controle. É frequentemente combinado com frameworks como PyTorch ou TensorFlow para criar agentes que aprendem por tentativa e erro. Entre as aplicações, temos o treinamento de agentes autônomos, como robôs de navegação ou bots para jogos.

6.5 spaCy

É uma biblioteca moderna e eficiente para PLN, com suporte a várias línguas e tarefas como análise sintática, reconhecimento de entidades e vetorização de texto. Aplicação: construção de agentes de linguagem que interpretam e processam dados textuais.

6.6 NLTK (Natural Language Toolkit)

É uma biblioteca robusta e bem documentada para PLN, com ferramentas para tokenização, análise gramatical, e modelagem de linguagem. Entre as aplicações, temos: desenvolvimento de agentes que precisam manipular e analisar texto de forma detalhada.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

6.7 LangChain

O LangChain é interessante no contexto de agentes inteligentes, especialmente quando falamos de agentes que integram modelos de linguagem, como GPT ou Llama, para interagir com usuários e realizar tarefas complexas.

O LangChain é uma ferramenta projetada para construir aplicações baseadas em modelos de linguagem de forma modular e eficiente, facilitando a integração de funcionalidades como busca em dados externos, raciocínio lógico e interação com APIs.

6.8 Hugging Face Transformers

Uma biblioteca que facilita o uso de modelos de linguagem de ponta, como GPT (Generative Pre-trained Transformer) ou Transformer Pré-Treinado Generativo, é um modelo criado pela OpenAI projetado para geração de texto e tarefas de linguagem natural. Ele utiliza uma arquitetura de Transformer unidirecional, isso significa que prevê a próxima palavra com base nas palavras anteriores.

Também trabalha bem com a BERT (Bidirectional Encoder Representations from Transformers), desenvolvido pelo Google. Projetado para entender o contexto de uma palavra considerando todo o contexto da frase, ou seja, analisando as palavras que vêm antes e depois da palavra-alvo.

E temos o “RoBERTa” (Robustly Optimized BERT Pretraining Approach) que é uma melhoria do BERT, desenvolvida pela Meta (Facebook), que ajusta o treinamento do modelo para torná-lo mais robusto e eficiente. Ele é baseado na mesma arquitetura do BERT, mas com mudanças importantes no treinamento. Aplicação: treinamento e ajuste fino de agentes que lidam com grandes volumes de texto e precisam de respostas altamente precisas e contextuais.

Todas essas bibliotecas possuem nuances que as diferenciam em:

- Na forma de tratar e ajustar os hiperparâmetros de forma mais agressiva, que otimiza o desempenho, ou mais suave, que não otimiza tanto o desempenho.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Na maneira de modelar a máscara de linguagem, onde algumas palavras em uma frase são “mascaradas” e o modelo tenta prever quais são ou se remove a tarefa de previsão de próxima sentença, focando exclusivamente na modelagem de máscara de linguagem.
- Se utiliza conjuntos de dados maiores e mais tempo de treinamento ou menores e menos tempo de treinamento.

6.9 OpenCV

Uma biblioteca de visão computacional popular e altamente otimizada, com suporte para processamento de imagens e vídeos. Aplicação: agentes que precisam interpretar informações visuais, como detecção de objetos e rastreamento.

6.10 YOLO (You Only Look Once)

É um modelo de aprendizado profundo otimizado para detecção de objetos em tempo real. Entre as aplicações, temos: agentes que precisam identificar e rastrear objetos rapidamente, como drones e câmeras de segurança.

6.11 Resumo comparativo

Enquanto o TensorFlow e o PyTorch são mais voltados para o treinamento de modelos de aprendizado profundo, o LangChain foca na implementação prática de agentes usando modelos pré-treinados, como o GPT ou o Llama.

O Gym é orientado para aprendizado por reforço e simulações, enquanto o LangChain é ideal para aplicações baseadas em linguagem e tarefas estruturadas.

O LangChain utiliza modelos como o GPT ou o Llama como um de seus componentes principais, mas oferece uma infraestrutura adicional para criar aplicações práticas e integrá-las a fluxos de trabalho mais amplos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

Essa análise aprofundada dos agentes de IA demonstra o papel essencial dessa tecnologia na resolução de problemas complexos e na automação de processos em diversas áreas.

Desde os agentes reflexivos simples, que reagem a condições imediatas, até sistemas multiagentes que integram diversas funções, a versatilidade desses modelos tem sido um diferencial para aplicações práticas.

Graças à evolução tecnológica que possibilitou não apenas a interação eficiente com ambientes físicos e digitais, temos também a adaptação a mudanças imprevisíveis, como observado nos agentes de aprendizado.

A implementação de agentes baseados em modelo e utilidade representa um avanço na capacidade de decisão autônoma, permitindo que sistemas de IA sejam mais responsivos e precisos. Exemplos como veículos autônomos e robôs industriais evidenciam como esses agentes lidam com ambientes dinâmicos e critérios múltiplos, equilibrando segurança, eficiência e conforto.

Além disso, os agentes orientados a metas e multiagentes ampliam ainda mais as possibilidades de inovação, possibilitando decisões estratégicas em tempo real e a colaboração eficiente entre componentes autônomos.

No entanto, a eficácia desses sistemas está intrinsecamente ligada ao uso de ferramentas avançadas como PyTorch, TensorFlow, e Llama, além de algoritmos robustos como Dijkstra para planejamento de rotas. Esses recursos são populares para superar os desafios de treinamento, de otimização e de implementação prática, garantindo que os agentes possam operar em larga escala e com altos níveis de precisão.

A capacidade de aprendizado e adaptação dos agentes é outra característica central para o avanço da IA. Técnicas como aprendizado por reforço e não supervisionado permitem que os agentes identifiquem padrões complexos e ajustem seu comportamento com base em novas informações. Isso os torna adequados para ambientes dinâmicos, onde a previsão antecipada de todas as variáveis seria inviável.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vimos que as aplicações abrangem setores como saúde, transporte, educação e comércio, demonstrando como os agentes de IA têm potencial para transformar interações humanas e processos automatizados. Seja monitorando cultivos agrícolas ou otimizando fluxos de tráfego em cidades inteligentes, o impacto desses sistemas é amplamente positivo, contribuindo para eficiência operacional e melhorias na qualidade de vida.

Por fim, a crescente sofisticação dos agentes de IA exige um alinhamento entre inovação tecnológica e responsabilidade ética. A integração cuidadosa desses sistemas deve priorizar a segurança, a privacidade e o impacto positivo na sociedade, garantindo que o avanço da automação beneficie a todos de maneira equitativa.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

BIGGS, John. Esse robô consegue carregar caixas em escritórios com espaço reduzido. **Uol**, 20 de fevereiro de 2020. Disponível em: <https://gizmodo.uol.com.br/robo-carregar-caixas-escritorios/>. Acesso em: 10 dez. 2024.

KANG, Hwan Il; LEE, Byunghee; KIM, Kabil. Path planning algorithm using the particle swarm optimization and the improved Dijkstra algorithm. In: **2008 IEEE Pacific-Asia Workshop on Computational Intelligence and Industrial Application**. IEEE, 2008. p. 1002-1004.

MARIO Wonder World Wiki. Toad. **Fandom**, 2024. Disponível em: <https://mario-wonder-world.fandom.com/wiki/Toad>. Acesso em: 11 dez. 2024.

QING, Guo; ZHENG, Zhang; YUE, Xu. Path-planning of automated guided vehicle based on improved Dijkstra algorithm. In: **2017 29th Chinese control and decision conference (CCDC)**. IEEE, 2017. p. 7138-7143.

RUSSELL, Stuart; NORVIG, Peter. **Artificial intelligence: a modern approach**. Hoboken. 2020.

STATISTA. **Artificial Intelligence – Worldwide**. March 2024. Disponível em: <https://www.statista.com/outlook/tmo/artificial-intelligence/worldwide#market-size>. Acesso em: 10 dez. 2024.

SUTTON, Richard S.; BARTO, Andrew G. **Reinforcement learning**: An introduction. MIT press, 2018.

VASWANI, A. **Attention is all you need**. Advances in Neural Information Processing Systems, 2017.

WANG, Huijuan; YU, Yuan; YUAN, Quanbo. Application of Dijkstra algorithm in robot path-planning. In: **2011 second international conference on mechanic automation and control engineering**. IEEE, 2011. p. 1067-1069.

WOOLDRIDGE, Michael. **An introduction to multiagent systems**. John wiley & sons, 2009.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com